File: mL_algo_from_scratch/naive_bayes.py

Purpose: Naive Bayes Probabilistic Classifier (based on Bayes Theorem)

```
"""Simple Gaussian Naive Bayes classifier.

This version is intentionally written in a simple, easy-to-follow style
so it looks like code a second-year engineering student might write.

Key ideas (short):
- Fit: for each class compute prior, mean and variance for each feature.
- Predict: for a sample, compute Gaussian log-probability per feature,
  sum them to get class log-likelihood, add log-prior, pick the best class.
"""
import numpy as np
```

=== Classes ===

```
class GaussianNB:
    """Gaussian Naive Bayes (very small, educational implementation).

    Attributes are plain and readable:
    - classes: array of unique labels
    - priors: prior probability for each class
    - means: per-class mean (matrix: n_classes x n_features)
    - variances: per-class variance (same shape as means)
    """
```

Study Tip: Initialization sets up the state. Look for hyperparameters.

Logic Focus: Defined task specifically for '__init__'

```
    def __init__(self):
        self.classes = None
        self.priors = None
        self.means = None
        self.variances = None
```

Study Tip: This method usually trains the model. Check how dimensions are handled.

Logic Focus: Defined task specifically for 'fit'

```
    def fit(self, X, y):
        """Learn class priors, means and variances from training data.

        X: array-like, shape (n_samples, n_features) or (n_samples,) for 1D
        y: labels, shape (n_samples,)
        """
        X = np.asarray(X)
        if X.ndim == 1:
            # make X two-dimensional when there's only one feature
            X = X.reshape(-1, 1)
        y = np.asarray(y)

        # Find the unique classes and how many examples of each
        self.classes, counts = np.unique(y, return_counts=True)
        self.priors = counts / counts.sum()

        n_classes = len(self.classes)
        n_features = X.shape[1]

        # Prepare containers for mean and variance
        self.means = np.zeros((n_classes, n_features))
        self.variances = np.zeros((n_classes, n_features))

        # Fill means and variances for each class using simple loops
        for i, cls in enumerate(self.classes):
            X_c = X[y == cls]
            # compute mean and variance per feature for this class
            self.means[i, :] = X_c.mean(axis=0)
```

```
            # add a tiny value to variance for numerical stability
            self.variances[i, :] = X_c.var(axis=0) + 1e-6

        return self
```

Logic Focus: Defined task specifically for '_log_gaussian'

```
    def _log_gaussian(self, x, mean, var):
        """Compute the log of Gaussian PDF for a vector x element-wise and sum.

        Formula for one feature:
            log p = -0.5 * log(2*pi*var) - (x-mean)^2 / (2*var)
        We return the sum across features because of the Naive Bayes independence assumption.
        """
        # coefficient term
        coeff = -0.5 * np.log(2 * np.pi * var)
        # exponent term
        exponent = -((x - mean) ** 2) / (2 * var)
        # sum across features
        return np.sum(coeff + exponent)
```

Key Logic: Inference happens here. Ensure data is preprocessed identically to training.

Logic Focus: Defined task specifically for 'predict'

```
    def predict(self, X):
        """Predict class labels for rows in X."""
        X = np.asarray(X)
        if X.ndim == 1:
            X = X.reshape(-1, 1)
```

Key Logic: Inference happens here. Ensure data is preprocessed identically to training.

```
        predictions = []
        for x in X:
            # compute log-posterior for each class: log-likelihood + log-prior
            log_posteriors = []
            for i in range(len(self.classes)):
                log_lik = self._log_gaussian(x, self.means[i], self.variances[i])
                log_prior = np.log(self.priors[i])
                log_posteriors.append(log_lik + log_prior)

            # choose class with highest log-posterior
            best_index = int(np.argmax(log_posteriors))
```

Key Logic: Inference happens here. Ensure data is preprocessed identically to training.

```
            predictions.append(self.classes[best_index])
```

Key Logic: Inference happens here. Ensure data is preprocessed identically to training.

```
        return np.array(predictions)
```

Logic Focus: Defined task specifically for 'score'

```
    def score(self, X, y):
        """Return accuracy on provided data."""
        y = np.asarray(y)
```

Key Logic: Inference happens here. Ensure data is preprocessed identically to training.

```
        preds = self.predict(X)
        return float(np.mean(preds == y))
```